МИНИСТЕРСТВО ОБРАЗОВАНИЯ РФ
Федеральное государственное бюджетное образовательное
учреждение высшего профессионального образования
«Кубанский государственный университет»
(ФГБОУ ВО «КубГУ»)

Факультет компьютерных технологий и прикладной математики
Кафедра вычислительных технологий

МЕТОДИЧЕСКИЕ РЕКОМЕНДАЦИИ к лабораторным работам

по дисциплинам

Б1.В.03 «Агентная парадигма программирования»

Направление подготовки
01.04.02. **Прикладная математика и информатика**
Магистерская программа "Математическое и программное обеспечение вычислительных систем"

Б1.В.03 «Мультиагентные системы»

Направление подготовки
02.04.02. **Фундаментальные информатика и информационные технологии**
Магистерская программа "Компьютерные науки"

Квалификация (степень) выпускника – магистр
Форма обучения: очная

Краснодар – 2023 г.

Методические рекомендации
для выполнения лабораторных работ и разработки индивидуального проекта по курсам

«Мультиагентные системы», «Агентная парадигма программирования»

Автор: к.т.н. доцент Приходько Т.А

Средой разработки и существования МАС являются агентные платформы. Было разработано множество программных реализаций агентных платформ, каждая из которых имеет свои особенности, достоинства и недостатки. Вот лишь небольшой список из более чем ста доступных платформ, публикуемых на сайте организации AgentLink (European Coordination Action for Agent-based Computing [5]): JADE, FIPA-OS, AOS, ZEUS, KADOMA, NOMADS, ARA, AGLETS, GRASSHOPPER, TRACY, AJANTA, LEAP, JACK, SEMOA.

Многие из них успешно существуют в виде коммерческих проектов (таких как JACK) или проектов, позиционируемых как проекты с открытым исходным кодом (JADE, ZEUS и др.).

В 90-х годах возникла необходимость создания единых стандартов на разработку агентных систем. В этот период были основаны две организации MASIF (Mobile Agent System Interoperability Facility) и FIPA (Foundation of Physical Intelligent Agents). В результате их работы появились стандарт MASIF и стандарт FIPA [6], дающие рекомендации по созданию систем мобильных агентов и систем интеллектуальных агентов. Одной из наиболее популярных агентных платформ в настоящее время является платформа JADE (Java Agent DEvelopment Framework). Проект JADE разрабатывается компанией Telecom Italia Lab с 2000 г.

Основные цели практикума:

- Изучение архитектуры и назначения агентной платформы JADE.
- Практическое освоение среды JADE для создания агентных приложений и управления ими.
- Изучение примеров разработки простых агентных приложений.
- Практическое освоение базовых классов, поддерживающих платформу JADE: классов агента, поведения агента, взаимодействия между агентами, сервиса «желтых страниц».
- Приобретение навыков программирования агентов на основе базовых классов JADE.
- Приобретение навыков построения агентных приложений, агенты которых выполняют сложные переговоры, публикацию и поиск сервисов.

В процессе выполнения лабораторного практикума предлагается разрабатывать агентные приложения в порядке возрастания сложности:

1. Простой агент HelloWorldAgent. Реализация проекта Ping-pong. Знакомство с агентной платформой и утилитами (стандартными агентами) JADE.
2. «Торговля книгами» – приложение, в котором агенты выполняют переговоры, публикацию и поиск сервисов. ЛР№3 разбита на 2 части: а) Типы поведения агентов и б) Средства коммуникации агентов на примере проекта «Торговля книгами».
3. Доклад с презентацией "Пример реализации реальной МАС". Детали задания см. в лекциях.
4. **Индивидуальный проект 1**. Задача: Матчинг «Заказ – Ресурс» как пример отношения, наиболее часто встречающегося в сфере производства и логистики, для реализации которого требуется взаимодействие между агентами. (Отчет обязателен).
5. **Индивидуальный проект 2**. Задача 2. Моделирование коллективного взаимодействия нескольких групп агентов. (Отчет обязателен).
6. **Индивидуальный проект 3.** Организация межплатформенной мобильности агентов. Здесь требуется организовать удаленное взаимодействие агентов, возможно использование платформы Jade-Leap под Android. (Отчет обязателен).

Даты выполнения заданий озвучиваются на лекции. Просроченное задание оценивается на балл ниже. Задания на индивидуальные проекты выдаются после выполнения заданий 1-4.

Содержание

Установка и настройка JADE *4*

Лабораторная работа №1 (часть 1). Разработка простых Агентов **5**

Цели работы: **5**

1.ч1.1 Как создать агентное приложение **5**

1.ч1.2. Теоретические сведения об агентах *9*

ЗАДАНИЕ К ЛР №1.ч1 ..... **10**

Лабораторная работа №1 (часть 2). Изучение структуры агентной платформы JADE *11*

1.ч2.1. Архитектура агентной платформы JADE **11**

1.ч2.2 Архитектура и реализация платформы JADE **13**

ЗАДАНИЕ К ЛР№ 1.ч2 ..... **17**

Лабораторная работа №2. Типы поведения агентов JADE. Этапы разработки мультиагентной системы *18*

Цель: изучить виды поведения агентов на примере мультиагентной системы, построенной по принципу pub/sub. *18*

2.1. Разработка класса-агента, JADE-класс **18**

2.2. Идентификаторы агента **19**

2.3. Реализация поведения агента **20**

2.4. Типы поведений агента **21**

2.5. Планирование операций в заданных временных точках **22**

2.6. Режимы работы агентов в приложении «Торговля книгами» **23**

2.6.1. Поведение агента Book-buyer 23

2.6.2. Поведение агента Book-seller 23

2.7. Средства коммуникации агентов **24**

2.7.1. Система отправки сообщений **24**

2.8. Взаимодействие агентов **26**

2.9. Сервис «желтых страниц» **31**

ЗАДАНИЕ К ЛР№2 ..... **35**

Установка и настройка JADE

1. **Скачать дистрибутив** с сайта проекта http://jade.tilab.com (сообщается, что сайт рискованный) Качаем **jadeAll.zip** после регистрации. WADE качать не нужно.
2. **Разархивировать** в папку, с коротким путем, содержащим только английские названия. (в переменную окружения PATH прописать необходимые пути к JAVA/bin и к JADE/bin)
3. **Создать новый java-проект в Eclipse или NetBeans**
4. **Подключить библиотеки Jade к проекту (рис.1.1).**



Для NetBeans:

Сначала создаем JAVA проект, который назовем, к примеру, JavaAgent. При создании проекта не забудьте выключить галочку на **Create Main Class**, это отключит создание главного класса, т.к. мы будем создавать все классы вручную.

Создаем комплект библиотек для подключения через меню Service (или Tools) – Libraries, далее «New Library» т.е. создаем свой набор библиотек, назовем его, к примеру, «Jade». Далее добавляем библиотеки JADE кнопкой «Add JAR/Folder» и указываем путь к библиотекам JADE. Это четыре библиотеки: http.jar, iiop.jar, jade.jar, jadeTools.jar, но в нашем случае достаточно одной: **jade.jar.**

Затем подключаем ее к проекту, для этого в свойствах проекта выбираем libraries и выбираем «Add Library…»
Результат наших действий (рис.1.1):

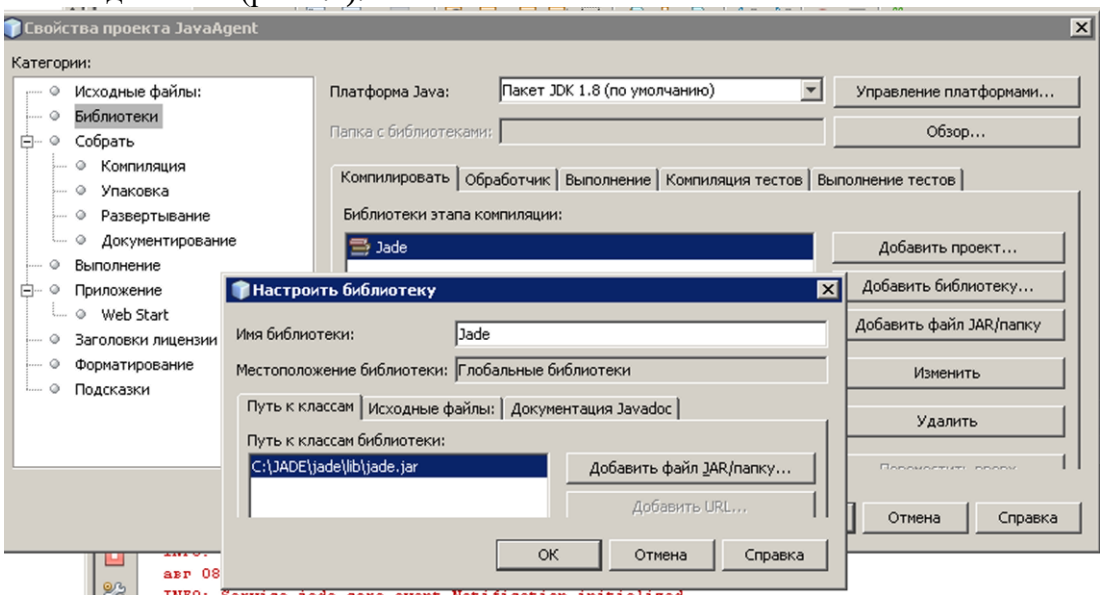


Рис. 1.1. Добавление библиотеки **jade.jar**

Для Eclipse:

Сначала необходимо создать Java-проект нашего агента в среде Eclipse и подключить JADE к этому проекту. Далее смотрите здесь: http://shcherbak.net/jadeeclipse/

Лабораторная работа №1 (часть 1). Разработка простых Агентов

Цели работы:

- Изучить архитектуру и назначения агентной платформы JADE.
- Освоить среду JADE для создания агентных приложений и управления ими.
- Изучить примеры разработки простых агентных приложений.

1.ч1.1 Как создать агентное приложение

Для примера мы создадим простейшую многоагентную систему с двумя агентами:

- один посылает сообщения всем,
- второй отвечает на это сообщение.

Для создания первого агента (Агент А), который будет отвечать на запросы, щелкаем правой кнопкой мыши по Source Packages и выбираем New -> Java Packages, назовем его ***A***, теперь создаем в этом пакете Java Class - его назовем ***AMain***. Делаем тоже самое и для Агента В (Пакет В, Java класс BClass). **(Листинги кодов для AMain и BClass указаны ниже по тексту.)**

Запустить платформу можно двумя способами: из командной строки или в среде разработки.

Вариант запуска из командной строки:

Перед тем как запустить нашего агента необходимо настроить параметры запуска проекта (рис.1.2). Это можно сделать, выбрав в свойствах проекта в свойстве «Run» - Выполнить.

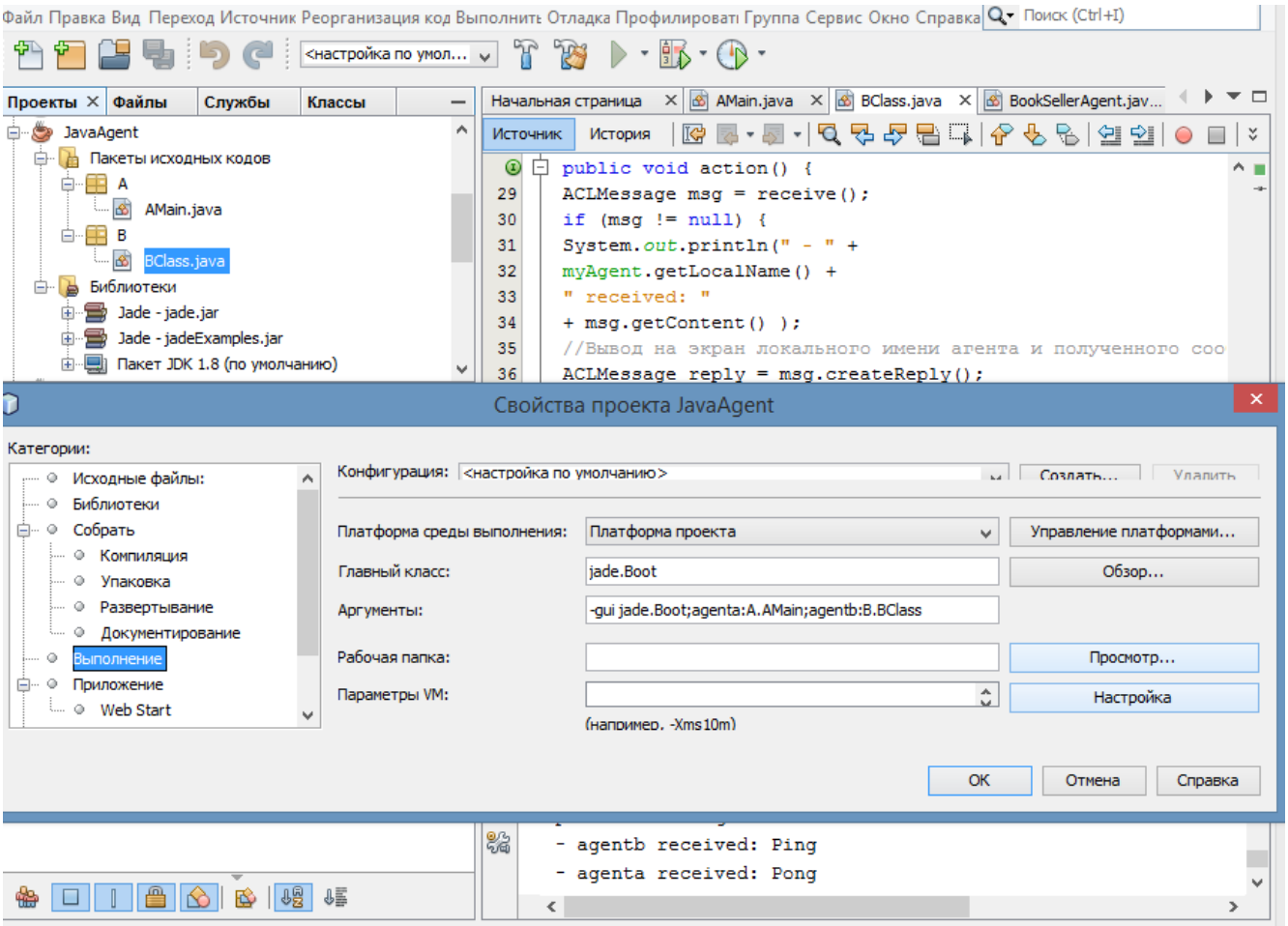


Рис. 1.2. Параметры запуска проекта

Вписываем главный класс как jade.Boot, а в аргументах указываем GUI системы - jade и тех агентов, которые будут запущены: agenta из пакета A, класса AMain и agentb из пакета B класса BClass В итоге получаем **(-gui jade.Boot;agenta:A.AMain;agentb:B.BClass)** – ***через точку с запятой и без пробелов***. Рядом запускается окно JADE (Рис 1.3). Щелкаем по AgentPlatforms и на папке с локальным именем платформы JADE и на папке ***MainContainer***, получаем то, что вы видите на рис.1.5., т.е. платформу с запущенными двумя пользовательскими агентами ***agenta*** и ***agentb***, а также служебными агентами платформы.

Если агенты не запускаются – проверьте совместимость версий Java и JADE.

Main Container (с графическим интерфейсом) можно также запустить консольной командой

java jade.Boot –gui

(не забудьте в переменную окружения PATH прописать необходимые пути к JAVA/bin и к JADE/bin).

Однако, запустив агента-сниффер, мы видим, что "переговоры" между ***agenta*** и ***agentb*** уже закончились и мы их не увидели. Поэтому придется стартовать нового агента из GUI (щелчок правой кнопкой мыши по названию контейнера Main-Container).

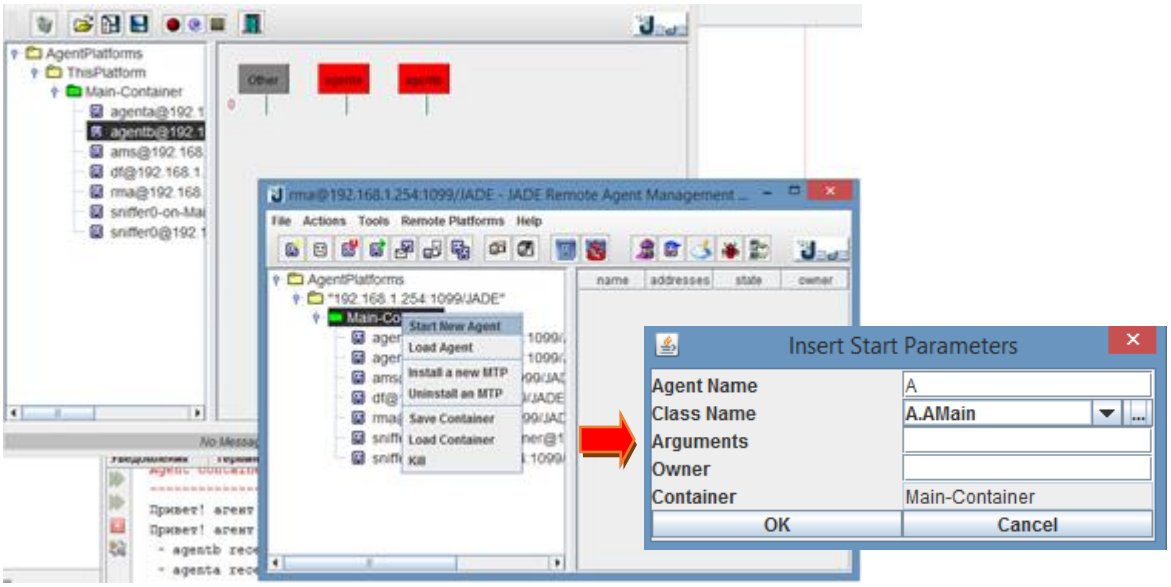


Рис.1.3. Старт нового агента из GUI

Попробуем отправить сообщение от имени нового агента A агентам B и agentb (рис 1.4.):

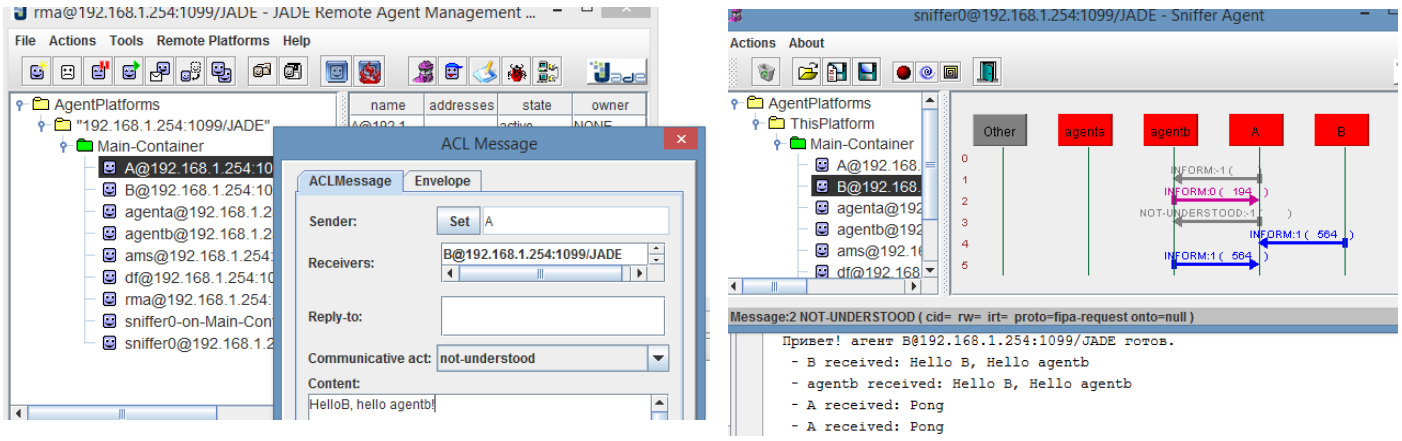


Рис.1.4. Отправка сообщений от A к B и agentb. И результат.

ПОДРОБНЕЕ О ПЛАТФОРМЕ JADE СМ. В ПРИЛОЖЕНИИ К ЛР№2.

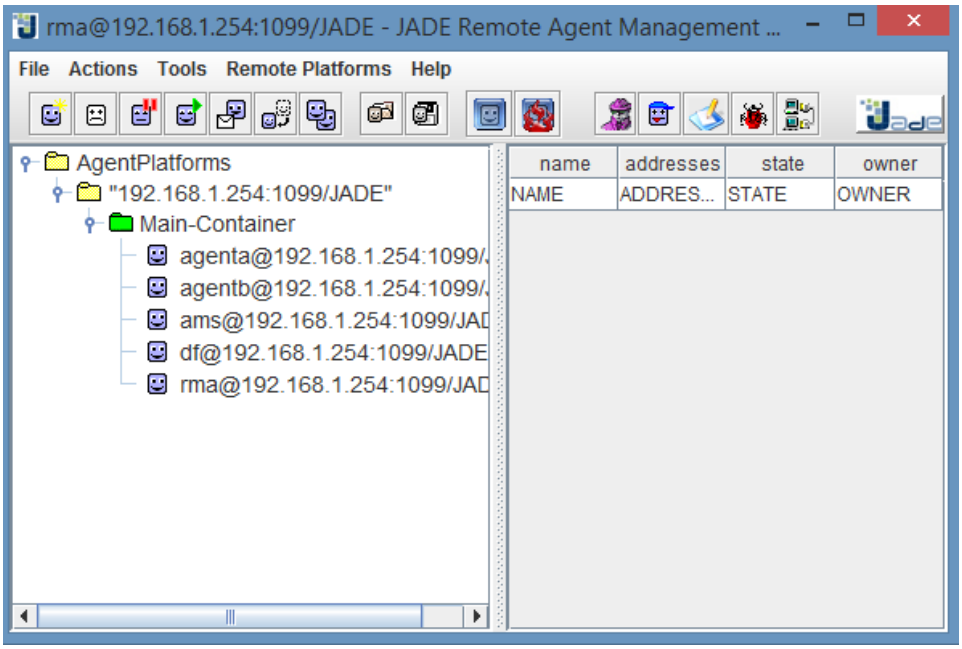


Рис.1.5. Запуск платформы JADE.

Каждый агент платформы JADE должен расширять базовый класс jade.core.Agent. В простейшем случае агент может быть реализован следующим образом:

Коды агентов:

```
Пакет A (Агент A):
/*
* To change this template, choose Tools | Templates and open
the template in the editor.*/
package A;

import jade.core.Agent;
import jade.core.AID;
import jade.domain.AMSService;
import jade.domain.FIPAAgentManagement.*;
import jade.core.behaviours.*;
import jade.lang.acl.*;

/**
*
* @author TRSteep
*/
public class AMain extends Agent {

public void setup() {
System.out.println("Привет! агент "+getAID().getName()+"
готов.");
addBehaviour(new CyclicBehaviour(this) // Поведение агента
исполняемое в цикле
{

public void action() {
    ACLMessage msg = receive();
    if (msg != null) {
```

```
    System.out.println( " - " + myAgent.getLocalName()+ "
received: "+ msg.getContent() );
     } //Вывод на экран локального имени агента и полученного
сообщения
     block();
//Блокируем поведение, пока в очереди сообщений агента не
появится хотя бы одно сообщение
   }
   });
   AMSAgentDescription [] agents = null;
      try {
           SearchConstraints c = new SearchConstraints();
            c.setMaxResults(new Long(-1));
            agents = AMSService.search(this, new
AMSAgentDescription(), c);
      } catch (Exception e) {
System.out.println( "Problem searching AMS: " + e);

e.printStackTrace();
}

      for(int i=0; i<4; i++) {
     AID agentID = agents[i].getName();
     ACLMessage msg = new ACLMessage(ACLMessage.INFORM);
     msg.addReceiver(agentID); // id агента которому отправляем
сообщение
      msg.setLanguage("English"); //Язык
      msg.setContent("Ping"); //Содержимое сообщения
      send(msg); //отправляем сообщение
}
}}
```

Пакет B (Агент B):

```
/*
* To change this template, choose Tools | Templates
* and open the template in the editor.
*/

package B;

import jade.core.Agent;
import jade.core.behaviours.*;
import jade.lang.acl.*;
/**
*
* @author TRSteep
*/
public class BClass extends Agent {

protected void setup() {
System.out.println("Привет! агент "+getAID().getName()+"
готов.");
addBehaviour(new CyclicBehaviour(this) {
```

```
public void action() {
ACLMessage msg = receive();
if (msg != null) {
System.out.println(" - " +
myAgent.getLocalName() +
" received: "
+ msg.getContent() );
//Вывод на экран локального имени агента и полученного
сообщения
ACLMessage reply = msg.createReply();
reply.setPerformative( ACLMessage.INFORM );
reply.setContent( "Pong");
//Содержимое сообщения
send(reply); //отправляем сообщения
}
block();
}
});
}}
```

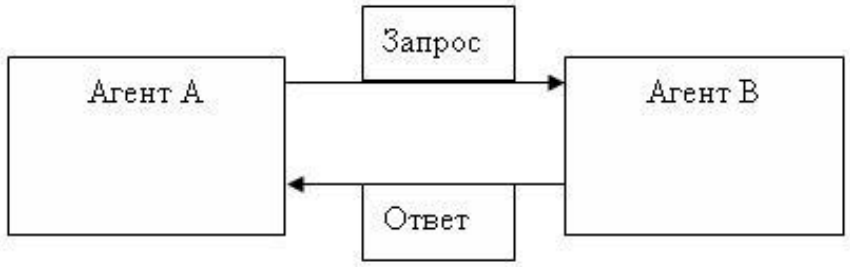


Рис 1.6. Диаграмма взаимодействия межу агентами

PingAgent - очень простой агент, дожидающийся сообщений. Если сообщение REQUEST с контентом PING получено, сообщение INFORM с подтверждением PONG посылается назад. Если же получено любое другое сообщение, назад отправляется сообщение NOT_UNDERSTOOD.

1.ч1.2. Теоретические сведения об агентах

Агент

Как видно из кода программ, с точки зрения платформы, агент JADE – обычный экземпляр определяемого пользователем Java-класса, который расширяет класс Agent, реализованный в платформе. Из этого следует, что базовый набор действий агента, необходимый для функционирования внутри платформы определен изначально.

Модифицируемость

Агент, как было сказано выше, имеет базовое наполнение как наследник класса Agent. Из каждого агента доступен общий функционал платформы:

- прием-передача сообщений и стандартные протоколы взаимодействий;
- планировщик поведений и набор стандартных поведений;

- менеджер управления жизненным циклом агента.

Прикладному разработчику требуется реализовать только «решатель» агента, анализирующий входные данные и выбирающий выходное воздействие. Если разработчик использует сторонние библиотеки, в которых реализованы «мозги» агента (JadeX, Jason и др.), тогда даже нет нужды напрямую работать с платформой, а логика работы агента описывается в терминах целей, планов их достижения и отдельных действий.

ЗАДАНИЕ К ЛР №1.ч1

1. Необходимо установить любую IDE с JAVA.
2. Скачать JADE, установить, выполнить настройку
3. Создать проект с двумя классами агентов A and B.
4. Протестировать программу. Опробовать ping– Агентов.
5. Сделать выводы.
6. Смоделируйте и поясните следующий результат (порядок появления агентов может быть другим):

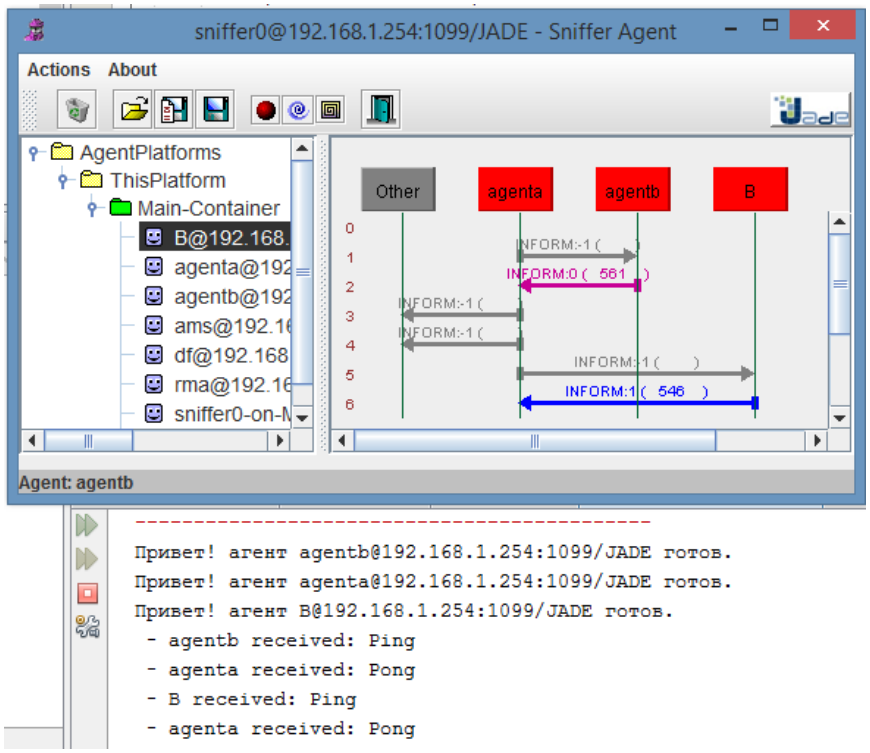


Рис. 1.7. Результаты моделирования в JADE

Лабораторная работа №1 (часть 2). Изучение структуры агентной платформы JADE

Одной из наиболее популярных агентных платформ в настоящее время является платформа JADE (**Java Agent DEvelopment Framework**). Проект JADE разрабатывается компанией Telecom Italia Lab с 2000 г. [7].

Агентная платформа JADE является типичным Middleware, т.е. программным обеспечением (ПО) промежуточного уровня (Рис. 2.1), представляющим собой набор средств для создания и управления системой с множеством агентов [8].

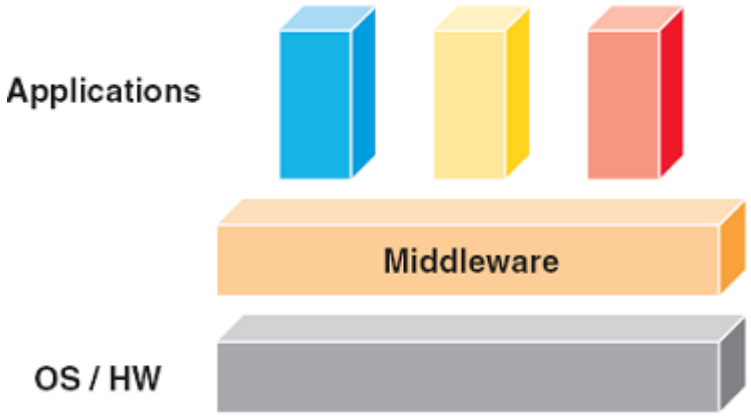


Рис. 2.1. Middleware JADE

1.ч2.1. Архитектура агентной платформы JADE

Платформа разработки мультиагентных систем JADE включает в себя динамическую среду, где могут «жить» JADE агенты; библиотеку классов, которую программисты могут использовать для разработки собственных агентов; набор графических инструментов, позволяющих управлять активностью запущенных агентов.

Включает в себя:

- среду выполнения агентов. Агенты регистрируются и работают под управлением среды;
- библиотеку классов, которые используются для разработки агентных систем;
- набор графических утилит для администрирования и наблюдения за жизнедеятельностью активных агентов.

Программная среда JADE подключается к любому проекту на языке Java.

JADE предоставляет программисту – разработчику агентных систем следующий набор средств [9]:

- ***FIPA-compliant Agent Platform*** – агентную платформу, основанную на стандарте FIPA и включающую обязательные типы системных агентов, которые автоматически активируются при запуске платформы.
- ***Distributed Agent Platform*** – распределенную агентную платформу, которая может использовать несколько компьютеров (узлов), причем на каждом узле запускается только одна Java Virtual Machine. Агенты исполняются как Java-потоки. Для доставки сообщений между агентами, в зависимости от их местонахождения, используется соответствующий транспортный механизм – Multiple Domains support – множество основанных на FIPA-спецификациях специализированных агентов, которые могут объединяться в федерацию, реализуя таким образом мультидоменную агентную среду.
- ***Multithreaded execution environment with two-level*** scheduling. Каждый JADE-агент имеет собственный поток управления, но он также способен работать в многопотоковом режиме. Java Virtual Machine проводит планирование задач, исполняемых агентами или одним из них.

- ***Object-oriented programming environment.*** Большинство концепций, свойственных FIPA-спецификации, представляются Java-классами, формирующими интерфейс пользователя.
- ***Library of interaction protocols.*** Использование стандартных интерактивных протоколов fipa-request и fipa-contract-net. Для того чтобы создать агента, который мог бы действовать согласно таким протоколам, разработчикам прикладных программ нужно только имплементировать специфические доменные действия, в то время как вся независимая от прикладной программы протокольная логика будет осуществляться системой JADE.
- ***Administration GUI.*** Простые операции управления платформой могут исполняться через графический интерфейс, отображающий активных агентов и контейнеры агентов. Используя GUI, администраторы платформы могут создавать, уничтожать, прерывать и возобновлять действия агентов, создавать иерархии доменов и мультиагентные федерации.

Платформа JADE написана на языке программирования Java с использованием Java RMI, Java CORBA IDL, Java Serialization и Java Reflection API. Она упрощает разработку мультиагентных систем благодаря использованию FIPA-спецификаций и инструментов (tools), которые поддерживают фазы исправления ошибок (debugging) и развертывания (deployment) системы. Эта агентная платформа может распространяться среди компьютеров с разными операционными системами, и ее можно конфигурировать через удаленный GUI-интерфейс. Процесс конфигурирования этой платформы достаточно гибкий. Единственным требованием такой системы является установка на компьютере Java Run Time требуемой версии.

Платформа JADE является распределенной и представляет собой набор контейнеров (Рис. 2.2).

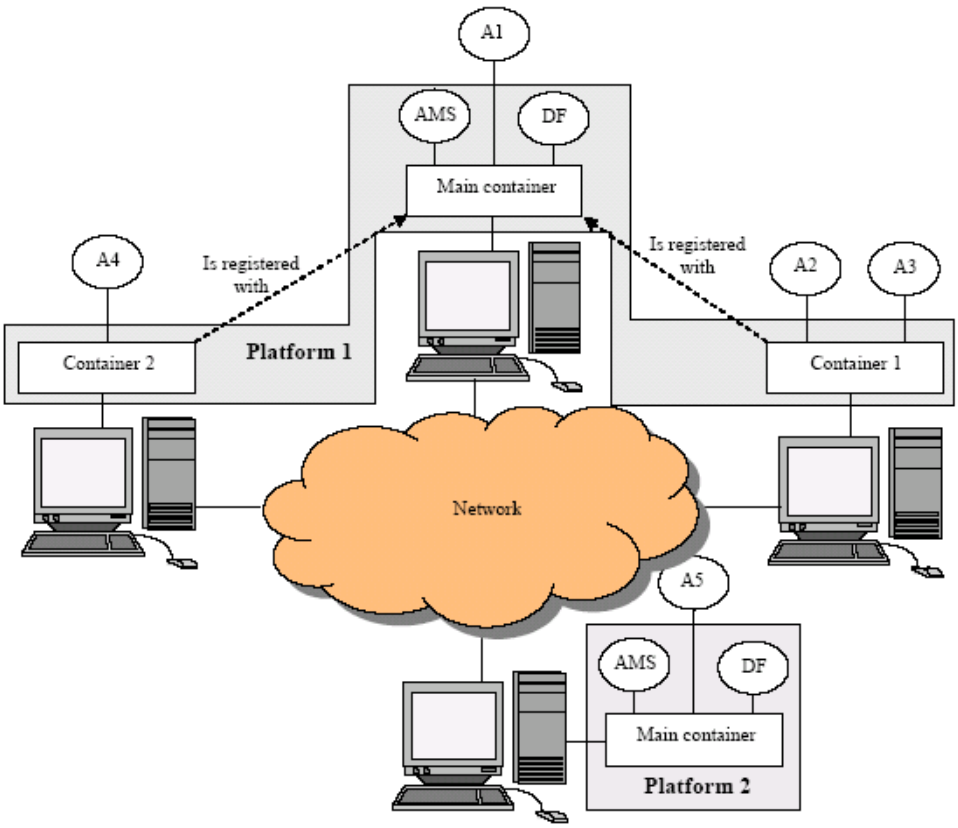


Рис. 2.2. Платформа JADE.

Контейнером называется динамическая среда исполнения мультиагентных приложений, в которой находятся агенты. Каждый контейнер может содержать несколько агентов. Набор активных контейнеров называется платформой. Один из контейнеров всегда является главным (Main container), все остальные контейнеры связываются с ним и регистрируются в момент запуска. Поэтому первым контейнером при старте платформы должен быть главный, а все остальные контейнеры должны быть «обыкновенными» контейнерами и должны заранее «знать», как найти главный контейнер, на котором они будут регистрироваться, т.е. должны иметь данные о хосте и порте. Другой главный контейнер, запущенный где-либо в сети, представляет

собой другую платформу, на которой могут зарегистрироваться новые обычные контейнеры. Рисунок 2.2 иллюстрирует эту концепцию на основе примера, показывающего две JADE-платформы, состоящие из трех и одного контейнера соответственно. JADE-агенты определяются с ***уникальными именами***. При условии, что они знают имена других агентов, они могут общаться, независимо от их фактического местонахождения: в общем контейнере (агенты A2 и A3), в разных контейнерах на одной платформе (агенты A1 и A2), или вообще на разных платформах (A4 и A5). Пользователю не обязательно знать, как работает динамическая среда JADE, но необходимо запускать ее перед началом выполнения своих агентов.

Агенты AMS и DF

Кроме возможности приема регистраций от других контейнеров, главный контейнер отличается от обычного контейнера тем, что содержит два специальных агента, автоматически запускаемых одновременно с контейнером:

- ***AMS (Agent Management System*** – система управления агентами) обеспечивает службу управления агентами, которая позволяет создавать и удалять агентов, а также содержит в себе пространство имен агентов. Имя агента является уникальным и имеет следующий формат: **<nickname>@<platform-name>.** Зная имена друг друга, агенты могут обмениваться сообщениями как внутри контейнера и платформы, так и между различными платформами.

- ***DF (Directory Facilitator – менеджер директорий)*** представляет собой службу «желтых страниц» (yellow pages), где агенты могут публиковать информацию о предоставляемых ими сервисах. С помощью ***DF*** агент может находить агентов, предоставляющих необходимые ему сервисы, и вступать с ними в переговоры. Внутри одной платформы может существовать несколько ***DF***, предоставляющих информацию о различных группах сервисов или о сервисах различных групп агентов. Использование ***DF*** описано далее в разделе Сервис «желтых страниц».

1.ч2.2 Архитектура и реализация платформы JADE

Под архитектурой системы понимается ее структура, то есть ее составляющие элементы, их внешние свойства и отношения-взаимосвязи между ними. Агент – это программа, обладающая независимым поведением и способная обмениваться сообщениями с другими агентами. Агент обладает набором поведений (функций, методов), для отработки заданной реакции на определенные события. Основной целью данной платформы является обеспечение существования и взаимодействия агентов.

Задачи агентной платформы заключаются в следующем:

• управление жизненными циклами агентов;

• эффективная передача сообщений между агентами.

Архитектура агентной платформы во многом определяется стандартом FIPA.

Выделяют следующие типы архитектурных структур:

• **структуры компонент-соединитель** – анализ работы JADE в период исполнения, оценка производительности и масштабируемости;

• модульные структуры – анализ конструкций в исходном коде, оценка расширяемости JADE;

• структуры распределения – анализ файловой организации JADE.

Remote management agent

Remote management agent (Рис.2.3) представляет собой графическую консоль для управления мультиагентным приложением. Позволяет создавать новые контейнеры, управлять агентами, создавать сообщения и запускать средства отладки.

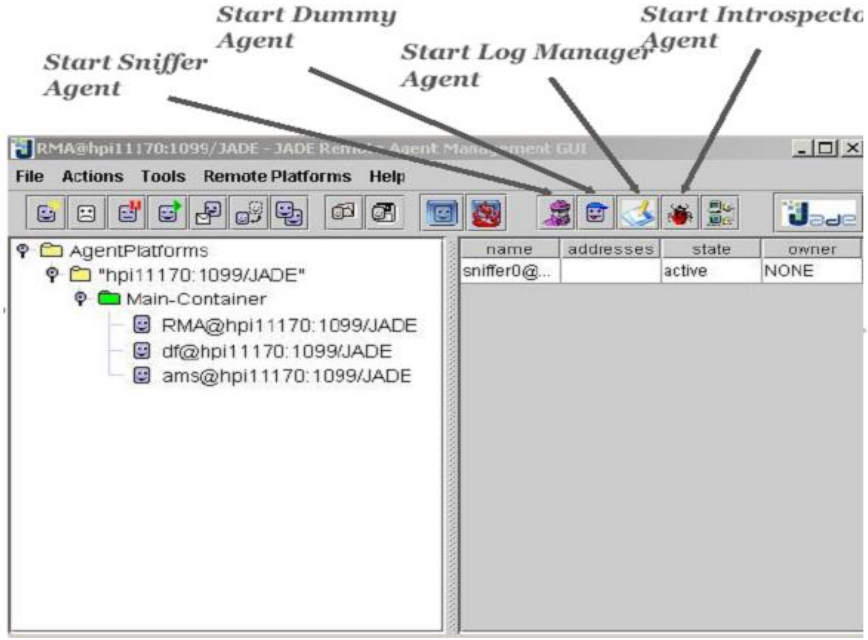


Рис. 2.3 - Remote management agent

Dummy agent

Dummy agent (Рис. 2.4) является графической утилитой, которая позволяет посылать и получать сообщения от имени определенного агента, а также сохранять и загружать очередь его сообщений (отправленных и полученных).

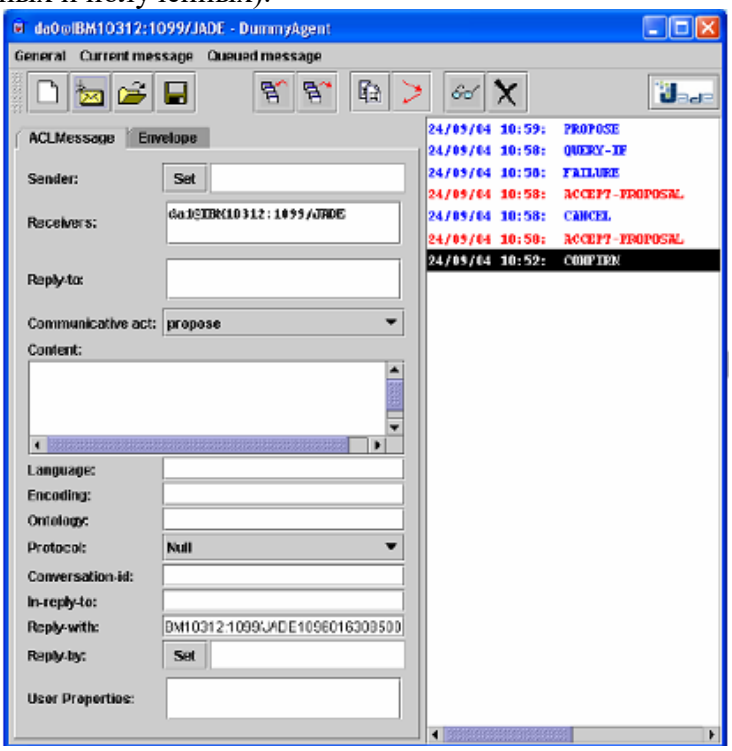


Рис. 2.4 – Dummy agent

Sniffer agent

Sniffer agent (Рис. 2.5) – это графическая утилита для просмотра потока сообщений между избранными агентами. Представляет обмен сообщениями в виде диаграмм последовательностей. Позволяет сохранять/загружать поток сообщений между агентами, а также просматривать содержимое сообщений.

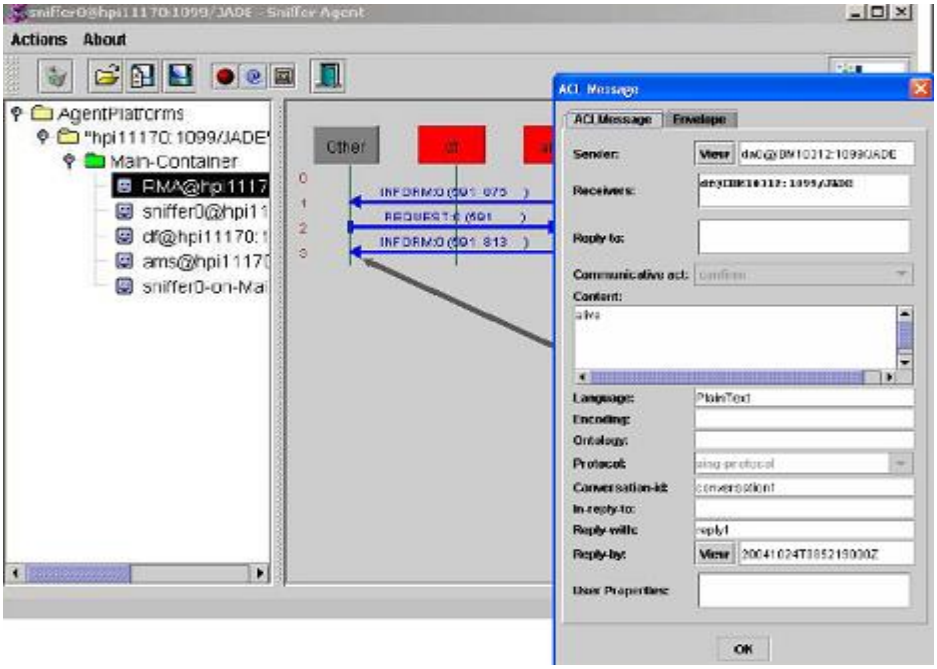


Рис.2.5 – Sniffer agent

Introspector agent

Introspector agent (Рис 2.6) – графическая утилита для просмотра внутреннего состояния агента. Позволяет контролировать жизненный цикл агента, просматривать очередь его сообщений, активные и выполненные поведения, а также запускать исполнение агента с задержками между операциями или по шагам. При этом «шагом» поведения агента считается исполнение метода action(), а не команда в коде языка Java.

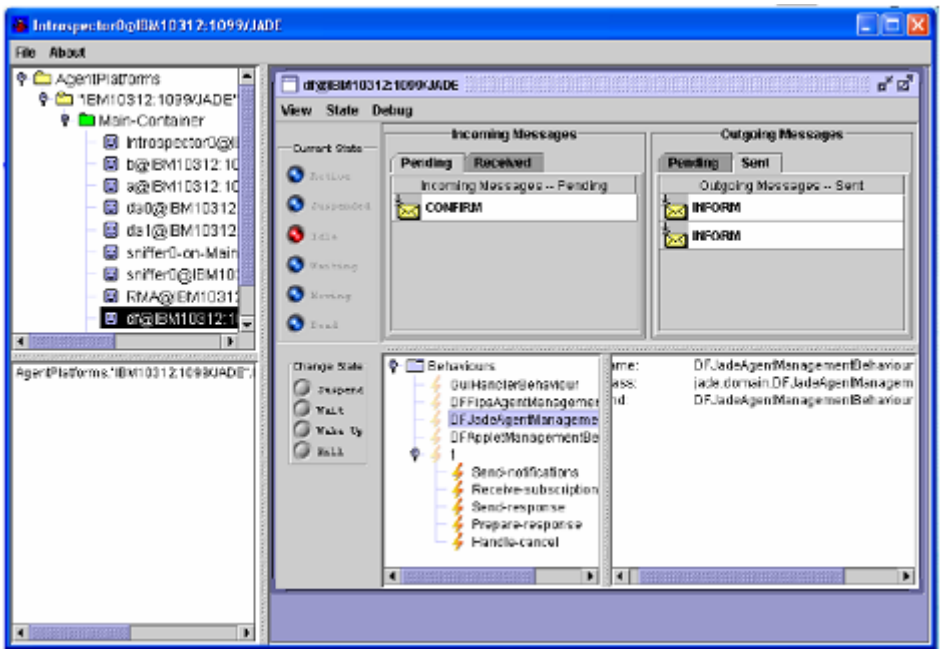


Рис. 2.6 – Introspector agent

Log manager agent

Log Manager agent – графическая утилита для отображения лога сообщений в процессе работы агентного приложения (исследуйте самостоятельно)

DF GUI

DF GUI (Рисунок 2.7) – рафическая утилита для визуализации желтых страниц. Позволяет регистрировать и удалять сервисы агентов, а также осуществлять поиск сервисов.

Этот JADE-агент, предназначенный для установки коммуникаций с удаленными клиентами позволяет поддерживать 50 параллельных подключений. Каждое входящее сообщение парсится, проверяется его адресат по листингу агентных имен, и передается агенту в пункт назначения.

JADE предлагает графический интерфейс агента DF (Directory Facilitator). Он позволяет сотрудничать с другими DF-агентами и контролировать (регистрировать, модифицировать, искать агентов по их характеристикам) всю сеть объединения DF-агентов. Этот тип агентов предлагает сервис “yellow pages” другим агентам. Они могут зарегистрировать свои услуги с помощью DF-агента или “попросить” его найти поддержку у других DF-агентов. Последние могут объединяться и создавать федерации агентов-фасилитаторов. Рис.2.7. показывает интерфейс таких агентов (добраться к немуможно через Tools агента AMSы).

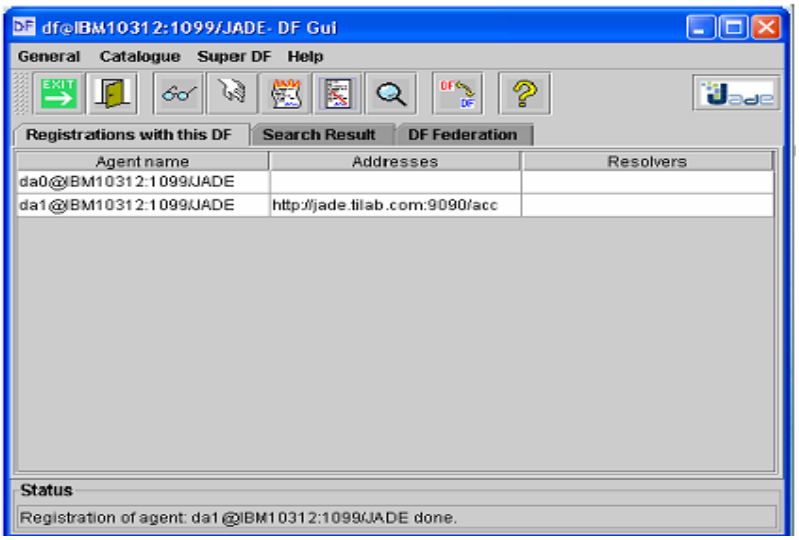


Рис. 2.7 – DF GUI

Все JADE-инструменты, кроме Dummy-Агента, наследуют от jade.tools. Класс ToolAgent, который обеспечивает способность получить уведомления события по общепринятому пути. На Рис.2.8 – представлена UML диаграмма классов инструментов JADE.

Каждый инструмент является Агентом, который расширяет jade.core.Agent – основной Agent класс. Это дает некоторые важные особенности и упрощения:

1. Жизненный цикл инструмента JADE может управляться точно, как любой другой агент платформы.
2. Возможность запроса главного класса Агента может быть использована для взаимодействия между инструментом и AMS (Agent Message System), в частности для получения уведомлений событий платформы.
3. Несколько экземпляров одного и того же инструмента может сосуществовать на той же платформе и даже в том же контейнере.

Класс jade.tools.ToolNotifier, порождающий вспомогательных агентов, требуемых для продвижения событий отправки-принятия и внутренних для агента событий к заинтересованным агентам, - является сам ToolAgent. Таким образом можно обнаружить момент, когда целевой агент

или агент наблюдателя уничтожается. Класс ToolNotifier жестко связан с ENS (EVENT NOTIFICATION SERVICE) и не рекомендован для прямого вызова программистами.

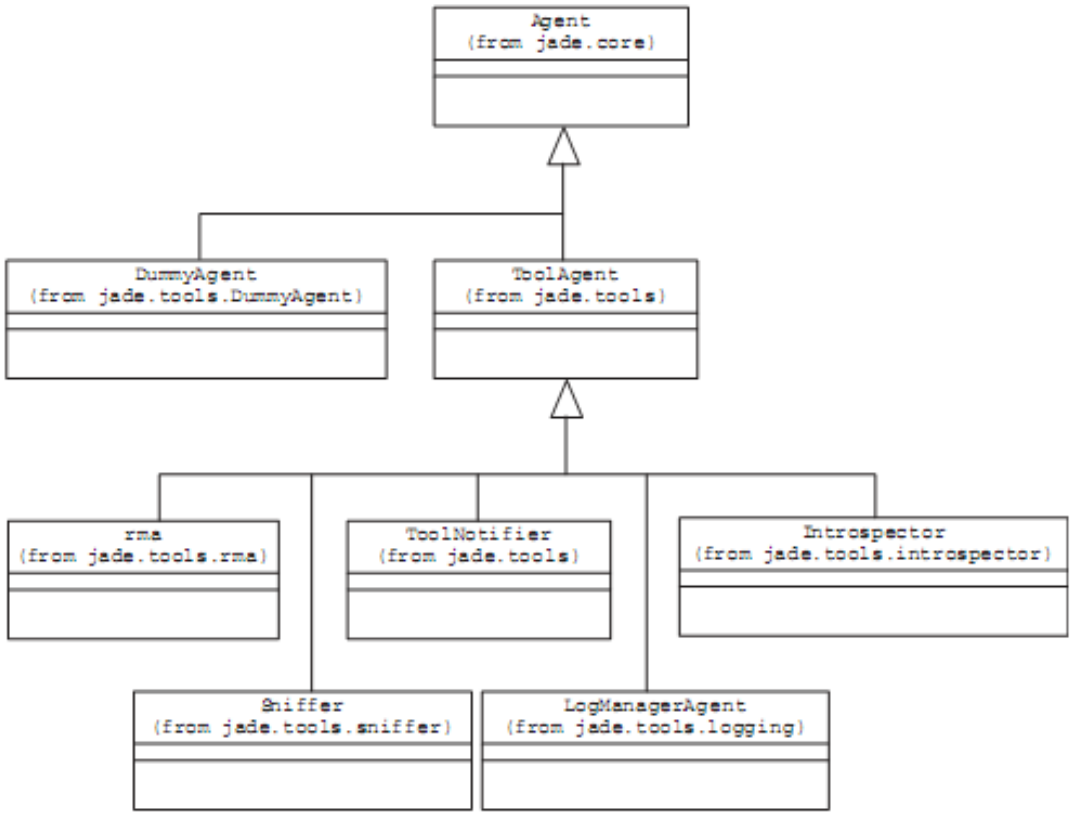


Рис. 2.8 – Диаграмма классов JADE-инструментов

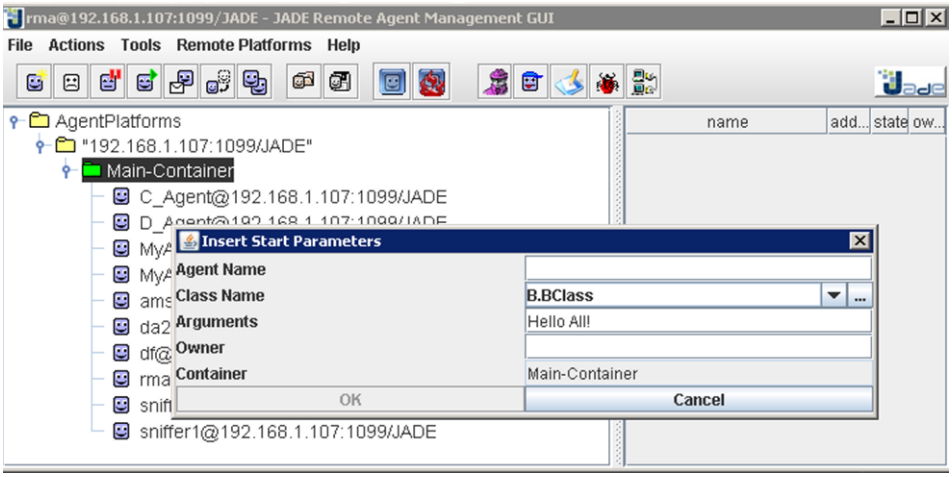


Рис. 2.9 – Создание нового агента

ЗАДАНИЕ К ЛРN 1.ч2

1. Создайте несколько агентов (Рис. 2.9), щелкнув правой кнопкой на MainContainer. (В пределах одной платформы требуются только имена агентов, например, MyAgentA)
2. Создайте также Dummy Agent. (при работе у него появляются два сообщения – синее и красное: синее –запрос, красное- ответ. И последнее сообщение находится сверху).
3. Отправьте несколько сообщений друг другу (с контентом ping и другим содержимым.)
4. Пронаблюдайте результат, в том числе в снифферe.
5. Сделайте выводы.
6. При защите ЛР предполагаются вопросы по теор. части из лекций и данной работы.

Лабораторная работа №2. Типы поведения агентов JADE. Этапы разработки мультиагентной системы

Цель: изучить виды поведения агентов на примере мультиагентной системы, построенной по принципу pub/sub.

ПРИМЕР "Торговля книгами "

Для иллюстрации пошаговой разработки агентного приложения на JADE рассмотрим пример "Торговля книгами"[1]. Сценарий, рассматриваемый в этом примере, включает агентов с "поведением", продающих книги и других агентов, покупающих книги от имени своих пользователей.

Каждый агент покупателя получает название книги для покупки ("Какая-то книга") в качестве аргумента командной строки и периодически запрашивает всех известных агентов торговца о наличии предложения этой книги. Как только предложение получено, агент покупателя оповещается об этом и выпускает заявку на поставку. Если более одного агента-торговца готовы обеспечить предложение, агент покупателя принимает лучший (меньшая цена). Купив заказанную книгу, агент покупателя заканчивает свою работу.

Каждый агент торговца имеет минимальный GUI, с помощью которого пользователь может вставить новые заглавия (и желательную цену) в местный каталог книг для продажи. Агенты продавцы находятся в постоянном ожидании запросов от агентов покупателей. Когда появляется заявка обеспечить предложение для книги, "торговцы" проверяют, есть ли запрошенная книга в их каталоге, и если да, то выдают положительный ответ и цену. Иначе они отказывают. Когда "торговец" получает заявку на поставку, он обслуживают эту заявку и удаляет запрошенную книгу из своего каталога.

Все проблемы, связанные с электронной оплатой, не рассматриваются в этом примере и не принимаются во внимание.

Полные листинги примера доступны в директории examples примеров JADE в пакете ***bookTrading.*** Структура программы показана на рис.2.10.

```
bookTrading/
        |
        |---src/
        |     |
        |     |---bookTrading/
        |              |
        |              |---buyer/
        |              |---seller/
        |---classes/
        |---lib/
        |      |
        |      |--- jade.jar
        |      |--- jadeTools.jar
        |      |--- http.jar
        |      |--- iiop.jar
        |      |---commons-codec/
        |---dist/
        |---build.xml
```

Рис.2.10. Структура программы «Торговля книгами»

2.1. Разработка класса-агента, JADE-класс

Для разработки класса агента нужно унаследовать класс Agent и реализовать функцию ***setup()*** – инициализатор агента.

```
import jade.core.Agent;

public class BookBuyerAgent extends Agent {
  protected void setup() {
    // Printout a welcome message
    System.out.println("Hello! Buyer-agent "+getAID().getName()+" is ready.");
  }
}
```

Пока наш класс без работы: реальная работа для агента обычно содержится в поведении - “behaviours”, как будет показано далее.

2.2. Идентификаторы агента

У каждого агента есть индивидуальный идентификатор – экземпляр класса ***jade.core.AID.*** Метод класса Агента ***getAID()*** позволяет получить идентификатор агента.

Объект типа ***AID*** включает глобально уникальное имя плюс ряд адресов. Как уже упоминалось в ЛР№2, имя агента в JADE имеет форму ***<nickname>@<platform-name>,*** таким образом, что агент под названием ***Peter***, живущий на платформе ***P1*** будет иметь ***Peter@P1***, как глобально уникальное имя. Адреса, входящие в ***AID***, - адреса платформы, в которой агент живет. Эти адреса только используются только в случае, когда агенту нужно связаться с другим агентом, "живущим" на другой платформе. Разработчикам нужно заботиться о них только в частных случаях, которые здесь мы обсуждать не будем.

Зная ***nickname*** агента, его ***AID*** может быть получен, как указано ниже:

AID id = new AID(nickname, AID.ISLOCALNAME);

Константа ISLOCALNAME указывает, что первый параметр представляет ***nickname*** (локальный для платформы), а не глобально уникальное имя агента.

Начнем разрабатывать класс агента-покупателя книг - `BookBuyerAgent`. Конечно же, агент-продавец BookSellerAgent будет похож, но на этом этапе будем полагать, что продавцы фиксированы, т.е. их имена априори известны и передаются каждому покупателю в качестве аргументов. Когда мы рассмотрим сервис «yellow pages», мы удалим эти ограничения.

Агенты могут получать при запуске аргументы, заданные из командной строки. Эти аргументы могут быть получены как массив *Object* с помощью метода *getArguments()* класса *Agent*. Агент *BookBuyerAgent* должен получить в качестве аргумента командной строки название книги, которую необходимо купить. В окончательной программе функция добавления известных продавцов выполняется в поведении с помощью сервиса желтых страниц.

```
import jade.core.Agent;
import jade.core.AID;

public class BookBuyerAgent extends Agent {

private String targetBookTitle; // The title of the book to buy

// The list of known seller agents
private AID[] sellerAgents = {new AID(“seller1”, AID.ISLOCALNAME),
new AID(“seller2”, AID.ISLOCALNAME)};

protected void setup() {
// Printout a welcome message
System.out.println(“Hello! Buyer-agent “+getAID().getName()+” is ready.”);

// Get the title of the book to buy as a start-up argument
Object[] args = getArguments();
if (args != null && args.length > 0) {
   targetBookTitle = (String) args[0];
   System.out.println(“Trying to buy “+targetBookTitle);
}
else {
// Make the agent terminate immediately
System.out.println(“No book title specified“);
doDelete();
```

```
}

protected void takeDown() {
// Printout a dismissal message
System.out.println("Buyer-agent "+getAID().getName()+" terminating.");
}
```

Аргументы командной строки задаются внутри скобок и разделяются подчеркиваниями:

```
-gui -agents buyer:bookTrading.BookBuyerAgent(The_Lord_of_the_ring)
```

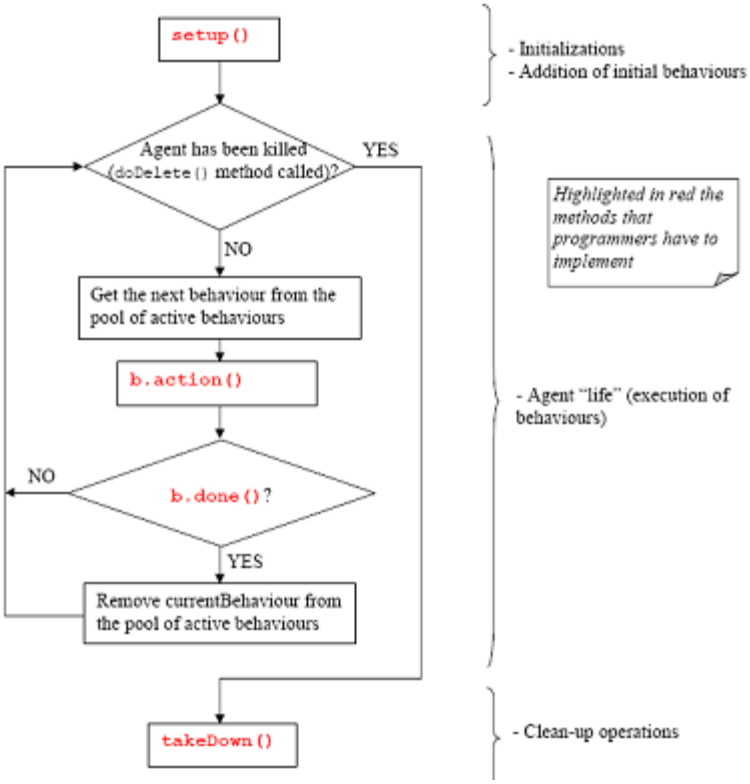
2.3. Реализация поведения агента

Фактическая работа, которую агент должен сделать, осуществляется в рамках "поведения" - 'behaviours'. Поведение представляет собой задачу, которую агент может выполнять и реализуется как объект класса ***jade.core.behaviours.Behaviour***. Поведение(я) должно быть добавлено к агенту посредством метода ***addBehaviour()*** класса **Agen**t. Поведение может быть добавлено в любое время, когда агент запускается (в методе ***setup()***).

Каждый класс расширяющий класс ***Behaviour*** должен реализовать два абстрактных метода: ***action() и done(). action()*** метод определяет операции, которые должны быть выполнены, когда «поведение исполняется». Метод ***done()*** возвращает `boolean` value, указывающее, была ли завершена работа поведения и должно ли это поведение быть удалено из пула исполняемых агентом поведений (работ).

Агент может выполнять несколько поведений одновременно. Важно учесть то, планировщик поведений не является упреждающим (pre-emptive (as for Java threads)), а является «кооперативным». Это значит, что, когда планировщик ставит поведение на исполнение, его метод ***action() исполняется до своего окончания.*** Т.е. программист должен сам определить, когда агент переключится с одного своего поведения на другое. С одной стороны, это создает некоторые трудности, но с другой, появляются и некоторые достоинства:

- поддерживает работу с одним потоком Java для агента (что является весьма важным в условиях ограниченных ресурсов, какими обладают, например, сотовые телефоны);
- обеспечивает более быстрое переключение между режимами работы, чем переключение между потоками Java;
- устраняет проблемы синхронизации параллельных режимов работы для доступа к одним и тем же ресурсам (что ускоряет производительность), т.к. все режимы выполняются одним и тем же потоком Java;
- позволяет сохранять состояние агента в постоянном хранилище для последующего возобновления (сохранность агента) или передачи его другому контейнеру для удаленного исполнения (мобильность агента).



Жизненный цикл агента представлен на Рис 2.11.

При создании агента вызывается метод ***setup(),*** в котором происходит инициализация агента. Далее в бесконечном цикле выбирается следующий режим поведения из пула активных и

исполняется. Затем, в зависимости от типа режима, он либо удаляется из пула активных, либо остается в нем и выполняется снова, когда до него дойдет очередь. При отсутствии активных «поведений» (например, если агент ждет сообщение) поток агента «засыпает» (что позволяет экономить ресурсы процессора). При удалении агента вызывается метод ***takeDown().***

В соответствии с описанным механизмом планирования, режим, представленный ниже, не позволяет запустить другой режим, т.к. его метод ***action()*** никогда не завершается.

```
public class OverbearingBehaviour extends Behaviour {
public void action() {
while (true) {
// do something}
}
public boolean done() {
return true; } }
```

2.4. Типы поведений агента

Различают три простых типа поведения агентов.

1) **Одноразовый *(one-shot)* режим**, у которого метод ***action*()** выполняется только один раз, после чего режим становится неактивным. ***jade.core.behaviours.OneShotBehaviour*** уже реализует метод ***done(),*** возвращающий значение ***true***, и может быть легко модифицирован для реализации одноразового поведения.

```
public class MyOneShotBehaviour extends OneShotBehaviour {
public void action() {
// perform operation X
}}
```

Операция X будет выполнена только один раз.

2) **Циклический *(cyclic)*** режим, который никогда не завершается и всегда продолжает оставаться активным. Метод ***action()*** выполняет одинаковые операции каждый раз, когда он вызывается. ***jade.core.behaviours.CyclicBehaviour*** уже реализует метод ***done(),*** возвращающий значение ***false***, и может быть легко модифицирован для реализации циклического поведения.

```
public class MyCyclicBehaviour extends CyclicBehaviour {
public void action() {
// perform operation Y
} }
```

Операция Y выполняется многократно, до завершения работы агента, выполняющего циклическое поведение.

3) **Общий *(generic)* режим**, который выполняет различные операции в зависимости от значения некоторой переменной. Режим остается активным до тех пор, пока выполняется условие, заданное в методе ***done().***

```
public class MyThreeStepBehaviour extends Behaviour {
private int step = 0;
public void action() {
switch (step) {
   case 0:
  // perform operation X
  step++;
  break;
```

```
  case 1:
  // perform operation Y
  step++;
  break;
  case 2:
// perform operation Z
  step++;
  break;
  }

public boolean done() {
return step == 3;
}}
```

Операции X, Y и Z выполняются одна за другой, а затем поведение завершается.

JADE предусматривает возможность комбинации простых режимов работы агента для создания более сложных режимов, о них даны некоторые сведения в лекциях.

2.5. Планирование операций в заданных временных точках

JADE предлагает два готовых класса (в пакете ***jade.core.behaviours***), с помощью которых можно легко реализовывать режимы работы, выполняющие определенные операции в заданные моменты времени.

1) Режим ***WakerBehaviour***, у которого методы ***action()*** и ***done()*** реализованы таким образом, что абстрактный метод ***handleElapsedTimeout()*** выполняется по истечении заданного интервала времени (указанного в конструкторе). После исполнения метода ***handleElapsedTimeout()*** поведение завершается.

```
public class MyAgent extends Agent {
protected void setup() {
System.out.println("Adding waker behaviour");
addBehaviour(new WakerBehaviour(this, 10000) {
protected void handleElapsedTimeout() {
// perform operation X
}
} );
}
```

Операция X выполняется через 10 секунд после вывода на экран сообщения «Adding waker behaviour».

2) Режим ***TickerBehaviour***, у которого методы ***action()*** и ***done()*** реализованы таким образом, что абстрактный метод ***onTick()*** выполняется многократно, через определенный интервал времени (указанный в конструкторе). Режим ***TickerBehaviour*** никогда не завершается.

```
public class MyAgent extends Agent {

    protected void setup() {
    addBehaviour(new TickerBehaviour(this, 10000) {
    protected void onTick() {
    // perform operation Y
    }
    } );
}
```

Операция Y выполняется периодически каждые 10 секунд.

2.6. Режимы работы агентов в приложении «Торговля книгами»

В соответствии с описанием основных типов поведения агентов, проанализируем поведение агентов ***Book-buyer*** и ***Book-seller*** в приложении «Торговля книгами».

2.6.1. Поведение агента Book-buyer

Агент ***Book-buyer*** должен периодически запрашивать у агента продавца книгу, которую ему было поручено купить. Этого можно достичь, если модифицировать режим ***TickerBehaviour*** так, что каждый «тик» добавляет другой режим, который фактически выполняет запрос к агенту-продавцу. Метод ***setup()*** класса ***BookBuyerAgent*** необходимо изменить следующим образом.

```
    protected void setup() {
// Printout a welcome message
System.out.println("Hello! Buyer-agent "+getAID().getName()+" is ready.");
// Get the title of the book to buy as a start-up argument

     Object[] args = getArguments();

     if (args != null && args.length > 0) {//Если есть название книги
          targetBookTitle = (String) args[0];
          System.out.println("Trying to buy "+targetBookTitle);
// добавить TickerBehaviour, которое организует запрос к «продавцу» каждую
   минуту
          addBehaviour(new TickerBehaviour(this, 60000) {

               protected void onTick() {
               myAgent.addBehaviour(new RequestPerformer());
               }
          } );
     }
     else {
     // Make the agent terminate
     System.out.println("No target book title specified");
     doDelete();
     }
```

Обратите внимание на использование защищенной переменной ***myAgent***: каждый режим (поведение) содержит указатель на агента, который его исполняет. Режим поведения ***RequestPerformer***, фактически выполняющий запрос к агенту-продавцу, будет описан в ЛР№3б - это класс, где будет обсуждаться взаимодействие между агентами.

2.6.2. Поведение агента Book-seller

Каждый агент ***Book-seller*** ожидает прихода запросов от агентов покупателей и обслуживает их. Запросы могут быть различных типов:

- Прием заявок от покупателей (OfferRequestsServer).
- Оформление сделки (*PurchaseOrdersServer*).
- Добавление книги в каталог.

Первые два рассмотрим после изучения взаимодействия агентов.

Для реализации запросов различных типов агенту ***Book-seller*** необходимо задать два циклических поведения: одно для обслуживания заявок, а другое для обслуживания заказов.

Кроме того, необходимо задать агенту ***Book-seller*** одноразовое поведение, реализующее *обновление каталога книг*, доступных для продажи в тех случаях, когда пользователь добавляет новую книгу из GUI. Класс ***BookSellerAgent*** может быть реализован следующим образом (классы ***OfferRequestsServer*** и ***PurchaseOrdersServer*** будут описаны в ЛР№3б Класс «взаимодействие между агентами»).

```
import jade.core.Agent;
import jade.core.behaviours.*;
import java.util.*;
public class BookSellerAgent extends Agent {
// The catalogue of books for sale (maps the title of a book to its price)
      private Hashtable catalogue;
      // The GUI by means of which the user can add books in the catalogue
      private BookSellerGui myGui;

      protected void setup()
      {
      // Create the catalogue
      catalogue = new Hashtable();
      // Create and show the GUI
      myGui = new BookSellerGui(this);
      myGui.show();

      // Add the behaviour serving requests for offer from buyer agents
      addBehaviour(new OfferRequestsServer());
      // Add the behaviour serving purchase orders from buyer agents
      addBehaviour(new PurchaseOrdersServer());
      }

      protected void takeDown() {
      myGui.dispose();
      System.out.println("Seller-agent "+getAID().getName()+" terminating.");
      }

            /**
            This is invoked by the GUI when the user adds a new book for sale
            */
            public void updateCatalogue(final String title, final int price) {
            addBehaviour(new OneShotBehaviour() {
            public void action() {
            catalogue.put(title, new Integer(price));
            }
            } );
}
```

2.7. Средства коммуникации агентов

Цель: изучить способы взаимодействия агентов на примере мультиагентной системы BookTrading - продавец-покупатель книг.

2.7.1. Система отправки сообщений

Одной из наиболее важных функциональных возможностей, которую предоставляют JADE агенты, является возможность коммуникации между ними. Обмен сообщениями в JADE является асинхронным и осуществляется с помощью ***Message Transport System*** (MTS). Формат сообщения соответствует стандарту FIPA (см. лекции). При этом внутри платформы для ускорения работы сообщения пересылаются в виде Java-объектов, а при обмене между платформами (или с другими MAC) – в виде XML-строки. При отправке сообщения оно попадает в очередь получателя, о чем он уведомляется. Сообщение может быть получено из «почтового ящика» агента и обработано **любым из его режимов**. Для выбора нужного типа сообщения из очереди программист может использовать фильтры. Пример обмена сообщениями показан на Рис 2.12.

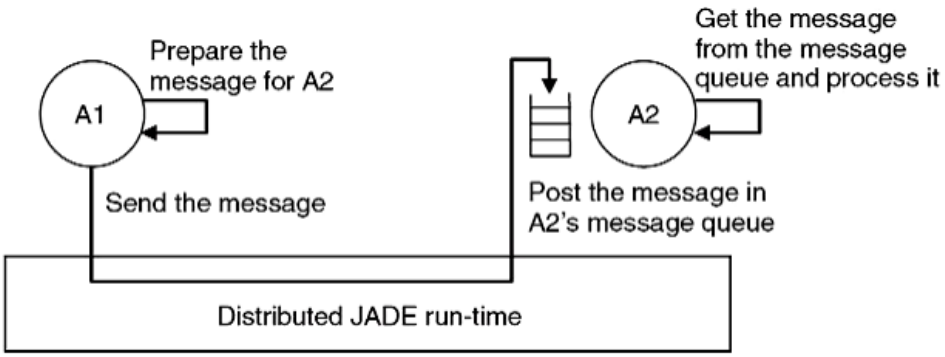


Рис 2.12. Обмен сообщениями между агентами.

Язык ACL

Сообщения, которыми обмениваются между собой агенты JADE, имеют формат языка ACL, определенного FIPA [4] как международный стандарт взаимодействия агентов. Этот формат включает следующие поля:

- отправитель сообщения;
- список получателей;
- коммуникативные действия («пожелания»), указывающие цель отправки сообщения. *Типы сообщений:*
  - **REQUEST** (запрос), если отправитель желает, чтобы получатель произвел некоторое действие,
  - **INFORM** (информирование), если отправитель желает, чтобы получатель был извещен о некотором факте,
  - **QUERY_IF**, если отправитель желает знать, достигнуто или нет заданное условие,
  - **CFP -** `Call for Proposal` (извещение о предложении),
  - **PROPOSE** (предложение),
  - **ACCEPT_PROPOSAL** (принятие предложения),
  - **REJECT_PROPOSAL** (отклонение предложения), если отправитель и получатель ведут переговоры, другие типы коммуникативных действий;
- содержание, т.е. фактическая информация, содержащаяся в сообщении (например, действия, которые будут выполняться по запросу REQUEST; факт, о котором отправитель желает информировать в сообщении INFORM и т.п.);
- язык, на котором представлено содержание (отправитель и получатель должны иметь возможность кодировать/декодировать выражения согласно синтаксису языка);
- онтология, т.е. словарь символов, используемых в содержании, и их значения (отправитель и получатель должны понимать смысл символов одинаково);

поля, используемые для управления несколькими параллельными переговорами, определяющие интервал времени для получения ответа, такие как conversation-id, reply-with, in-reply-to, reply-by. Сообщение в JADE реализуется как объект класса *jade.lang.acl.ACLMessage*, предоставляющего методы get(). Пример сообщения:

This is a simple example of a FIPA-ACL message with a *request* performative:

```
(request
    :sender (agent-identifier :name alice@mydomain.com)
    :receiver (agent-identifier :name bob@yourdomain.com)
    :ontology travel-assistant
    :language FIPA-SL
    :protocol fipa-request
    :content
      ""((action
          (agent-identifier :name bob@yourdomain.com)
           (book-hotel :arrival 15/10/2006
                       :departure 05/07/2002 ... )
      ))""
)
```

2.8. Взаимодействие агентов

Чтобы отправить сообщение другому агенту, необходимо заполнить поля объекта *ACLMessage* и вызвать метод *send()* класса *Agent*. Приведенный ниже код демонстрирует, как создать и отправить сообщение, информирующее агента по имени *Peter* о том, что сегодня идет дождь.

```
ACLMessage msg = new ACLMessage(ACLMessage.INFORM);
msg.addReceiver(new AID("Peter", AID.ISLOCALNAME));
msg.setLanguage("English");
msg.setOntology("Weather-forecast-ontology");
msg.setContent("Today it's raining");
send(msg);
```

Отправленные сообщения платформа JADE автоматически ставит в очередь доставленных сообщений агента-получателя. Агент может получить к ним доступ, вызвав метод *receive()*, который возвращает первое сообщение из очереди или null, если очередь пуста:

```
ACLMessage msg = receive();
if (msg != null) {
// Обработка сообщения
}
```

Посылка сообщений в приложении «Торговля книгами»

В приложении «Торговля книгами» используется сообщение **CFP** (`Call for Proposal`) для извещения о том, что агент покупателя Buyer-agent отправил агенту продавца Seller-agent запрос на приобретение книги. Сообщение **PROPOSE** содержит предложение продавца, а сообщение **ACCEPT_PROPOSAL** содержит заказ покупателя. Сообщение **REFUSE** агента продавца сообщает о том, что книги, запрошенной агентом покупателя, нет в каталоге. В обоих типах сообщений, отправляемых агенту покупателя, содержится название книги.

Содержанием сообщения PROPOSE должна быть цена на книгу. Ниже приведен пример, демонстрирующий, каким образом создается и оправляется сообщение CFP покупателем:

```
// Сначала в функции srtup считывается аргумент «название книги» - targetBookTitle
protected void setup() {
System.out.println("Hallo! Buyer-agent "+getAID().getName()+" is ready.");

            // Get the title of the book to buy as a start-up argument
            Object[] args = getArguments();
            if (args != null && args.length > 0) {
                  targetBookTitle = (String) args[0];
                  System.out.println("Target book is "+targetBookTitle);

                   …
      }}
```

```
// Затем рассылается запрос на заданную книгу всем агентам-продавцам:

private class RequestPerformer extends Behaviour {
…
// Message carrying a request for offer
ACLMessage cfp = new ACLMessage(ACLMessage.CFP);
for (int i = 0; i < sellerAgents.lenght; ++i) {
     cfp.addReceiver(sellerAgents[i]);
     }
   cfp.setContent(targetBookTitle);
…}
```

Блокирование режима работы агента для ожидания сообщения

Часто необходимо реализовать поведение агента, который обрабатывает сообщения от других агентов. В нашем случае это *OfferRequestsServer* and *PurchaseOrdersServer* – в каждом из них требуется обработка заявок покупателя. Такое поведение должно быть повторяющимся, а при каждой итерации необходимо проверять наличие новых сообщений и обрабатывать их. Методы аналогичны, рассмотрим реализацию *OfferRequestsServer*.

Реализация поведения агента-продавца – обработка заявки:

```
public class OfferRequestsServer extends CyclicBehaviour {
public void action() {
        ACLMessage msg = myAgent.receive();
        if (msg != null) {

            //Сообщение принято, обработаем его

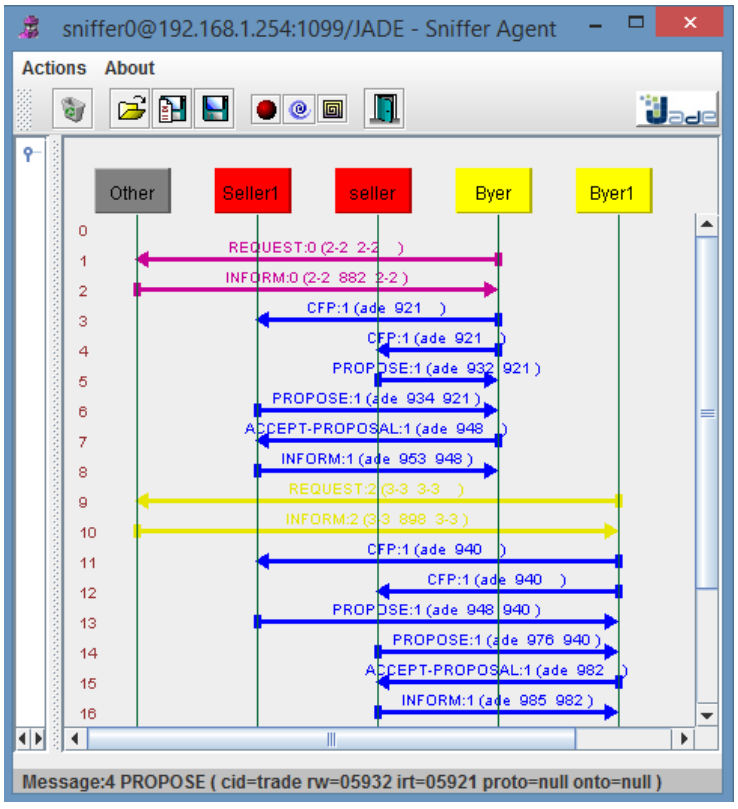
            String title = msg.getContent();
            ACLMessage reply = msg.createReply();
            Integer price = (Integer) catalogue.get(title);
             if (price != null) {

        // Запрашиваемая книга есть у продавца. Возвращаем ее цену

            reply.setPerformative(ACLMessage.PROPOSE);
            reply.setContent(String.valueOf(price.intValue()));
    } else {

            // Запрашиваемая книга отсутствует у продавца

            reply.setPerformative(ACLMessage.REFUSE);
           reply.setContent("not-available");
   }
    myAgent.send(reply);
}
        }
}
```



На рисунке можно увидеть последовательность сообщений:

В коде (выше) поведение *OfferRequestsServer* реализовано как внутренний класс класса *BookSellerAgent*. Это упрощает реализацию, т.к. обеспечивает прямой доступ к каталогу книг для продажи, но не является обязательным.

Метод *createReply ()* класса *ACLMessage* автоматически создает новую настройку *ACLMessageproperly* получателя и все поля, используемые для управления сеансом (*conversation-id, reply-with, in-reply-to*).

В соответствии с Рис.2.11, при добавлении поведения, описанного выше, поток агента входит в непрерывный цикл, что слишком нагружает процессор. Чтобы избежать этого, *метод action()* поведения *OfferRequestsServer* следует выполнять лишь

тогда, когда появится новое сообщение. Для этого можно использовать метод *block()* класса поведений. Этот метод помечает поведение как «заблокированное», так что агент не планирует следующих выполнений. Когда в очередь сообщений агента добавится новое сообщение, все заблокированные режимы снова становятся доступными для исполнения:

```
public void action() {
ACLMessage msg = myAgent.receive();
if (msg != null) { // Message received. Process it
...
}
else {block();
}
```

Приведенный выше код является типичным (и рекомендуемым) шаблоном для реализации приема сообщений внутри поведения агента.

Выбор сообщений с указанными характеристиками из очереди сообщений

Т.к. оба режима ***OfferRequestsServer*** и ***PurchaseOrdersServer*** являются циклическими, их метод ***action()*** которых начинается с вызова метода ***myAgent.receive()***, что порождает следующую проблему: как определить, что поведение ***OfferRequestsServer*** выбирает из очереди сообщений только сообщения, в которых содержится запрос о наличии книги, а поведение ***PurchaseOrdersServer*** – только сообщения, содержащие заказы на поставку?

Для разрешения этой проблемы нужно изменить код, указав соответствующие “шаблоны” при вызове метода ***receive().*** Когда шаблон указан, метод ***receive()*** возвращает первое сообщение (если оно есть), соответствующее данному шаблону, игнорируя все неподходящие сообщения. Подобные шаблоны реализованы как экземпляры класса ***jade.lang.acl.MessageTemplate***, который предоставляет набор методов, позволяющих просто и гибко создавать шаблоны.

Как было сказано выше *Посылка сообщений в приложении «Торговля книгами»*, для запроса о наличии книги используется сообщение **CFP**, и сообщение **PROPOSE** – для передачи предложения о заказе. Поэтому необходимо изменить метод *action()* класса *OfferRequestServer* так, чтобы вызов *myAgent.receive()* игнорировал все сообщения, кроме **CFP**.

```
public void action() {
//Этот шаблон призван выбрать только сообщения типа CFP
      MessageTemplate mt =
      MessageTemplate.MatchPerformative(ACLMessage.CFP);
      ACLMessage msg = myAgent.receive(mt) ;
      if (msg != null) {
      // CFP Message received. Process it
      ...
      }
      else {
      block();
      }
}
```

Выдержка из таблицы методов класса Class MessageTemplate:

static MessageTemplate	**MatchPerformative** (int value) This *Factory Method* returns a message template that matches any message with a given performative.
static MessageTemplate	**MatchProtocol** (java.lang.String value) This *Factory Method* returns a message template that matches any message with a given :protocol slot.
static	**MatchReceiver** (AID [] values)

MessageTemplate	This *Factory Method* returns a message template that matches any message with a given `:receiver` slot.

Сложные переговоры

Режим *RequestPerformer* (поведение агента *Bookbuyer*) представляет собой пример поведения, проводящего «сложные» переговоры. В данном случае, «переговоры» – это последовательность сообщений, которыми обмениваются два или более агентов с четко определенными причинными и временными отношениями. Поведение *RequestProposal* должно послать **CFP**-сообщение нескольким агентам - известным агентам-продавцам, получить обратно все ответы и, в случае, если получен хотя бы один ответ типа PROPOSE, позже послать сообщение ACCEPT_PROPOSAL агенту-продавцу, который сделал предложение с минимальной ценой, и получить обратно ответ INFORM. Всякий раз, когда происходит обмен сообщениями, необходимо определить управляющие поля в сообщениях, участвующих в обмене. Это позволит легко и однозначно создавать шаблоны, соответствующие возможным ответам.

```
import jade.core.AID;
import jade.core.behaviours.Behaviour;
import jade.lang.acl.ACLMessage;
import jade.lang.acl.MessageTemplate;

public class RequestPerformer extends Behaviour {
      private AID bestSeller; //агент-продавец с лучшей(наименьшей ценой)
      private int bestPrice; // лучшая цена
      private int repliesCnt = 0; // счетчик ответов от продавцов
      private MessageTemplate mt;
      private int step = 0;

      public void action() {
        switch (step) {
case 0:
        // отправим запрос на книгу всем продавцам
        ACLMessage cfp = new ACLMessage(ACLMessage.CFP);
        for (int i = 0; i < sellerAgents.length; ++i) {
          cfp.addReceiver(sellerAgents[i]);
       }
          cfp.setContent(targetBookTitle);
       cfp.setConversationId("book-trade");
       cfp.setReplyWith("cfp" + System.currentTimeMillis());
       myAgent.send(cfp);
       // подготовим шаблон для получения цен на книги
       mt = MessageTemplate.and(MessageTemplate
              .MatchConversationId("book-trade"), MessageTemplate
              .MatchInReplyTo(cfp.getReplyWith()));
      step = 1;
      break;
case 1:
      // получим ответы с ценами, либо отказами, если книги нет
      ACLMessage reply = myAgent.receive(mt);
      if (reply != null) {
            if (reply.getPerformative() == ACLMessage.PROPOSE) {
                  int price = Integer.parseInt(reply.getContent());
                    if (bestSeller == null || price < bestPrice) {
                          // это лучшее предложение
                            bestPrice = price;
                            bestSeller = reply.getSender();
                          }
      }
```

```
        repliesCnt++;
        if (repliesCnt >= sellerAgents.length) {
                // приняты ответы от всех продавцов
                step = 2;
        }
    } else {
          block();
    }
    break;
case 2:

 // отправим запрос на покупку агенту, предложившему лучшую цену
ACLMessage order = new ACLMes-sage(ACLMessage.ACCEPT_PROPOSAL);
order.addReceiver(bestSeller);
order.setContent(targetBookTitle);
order.setConversationId("book-trade");
order.setReplyWith("order" + System.currentTimeMillis());
myAgent.send(order);
mt = MessageTemplate.and(MessageTemplate
       .MatchConversationId("book-trade"), MessageTemplate
       .MatchInReplyTo(order.getReplyWith()));
step = 3;
break;
case 3:
reply = myAgent.receive(mt);
if (reply != null) {
         // принят ответ не предложение покупки
         if (reply.getPerformative() == ACLMessage.INFORM) {
                // покупка совершена
     System.out.println(targetBookTitle + " successfully purchased.");
                System.out.println("Price = " + bestPrice);
                myAgent.doDelete();
         }
         step = 4;
       } else {
            block();
              }
  break;
    }
}
public boolean done() {
        return ((step == 2 && bestSeller == null) || step == 4);
}
}
```

Обычно переговоры проходят по стандартной, заранее известной, схеме (протоколу). JADE предоставляет возможность реализовать переговоры по протоколу с помощью классов из пакета ***jade.proto***. Получение сообщений в режиме блокировки. Кроме метода ***receive()*** класс ***Agent*** предоставляет еще один метод для получения входящих сообщений – метод ***blockingReceive().*** Отличие этого метода заключается в том, что возврат из него не происходит пока не будет получено сообщение, то есть происходит блокировка агента (потока агента и, следовательно, его поведения). Аналогично, перегруженная версия метода ***blockingReceive***, принимающая в качестве параметра шаблон, блокируется до получения сообщения, удовлетворяющего используемому шаблону.



Рис 2.13. Диаграмма классов приложения «Торговля книгами»

2.9. Сервис «желтых страниц»

В приведенном примере было сделано допущение, что существует некое фиксированное множество агентов-продавцов (*seller1* и *seller2*) и каждый покупатель заранее узнает о них из строки аргументов (поле *AID[] sellerAgents* класса *BookBuyerAgent* в коде, приведенном выше (раздел 3.2 ЛР№3) *Передача аргументов агенту*). Далее описано, как использовать сервис «желтых страниц», предоставляемый платформой JADE, чтобы позволить агентам-покупателям динамически узнавать об агентах-продавцах, доступных в данный момент времени.

Агент DF

Агент *Directory Facilitator* (DF) – менеджер директорий (согласно спецификации FIPA) представляет службу желтых страниц (*yellow pages*), где агенты могут публиковать информацию о предоставляемых ими сервисах. С помощью DF агент может находить агентов, предоставляющих необходимые ему сервисы, и вступать с ними в переговоры. Внутри одной платформы может существовать несколько DF, например, предоставляющих информацию о различных группах сервисов или о сервисах различных групп агентов. Пример использования DF показан на Рис 4.3.

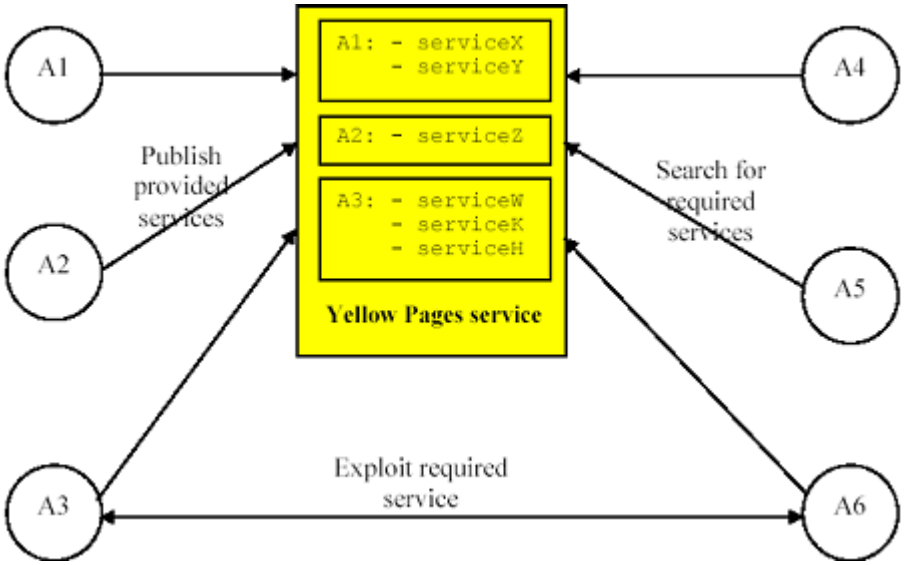


Рис.2.14. Сервис «желтых страниц».

Каждая платформа JADE по умолчанию поддерживает агента DF (его локальное имя «df»). Могут быть запущены и объединены несколько DF-агентов (включая те, что находятся в платформе по умолчанию), для обеспечения единого распределенного каталога «желтых страниц».

Взаимодействие с DF

Поскольку DF является агентом, имеется возможность взаимодействовать с ним стандартным образом, обмениваясь ACL-сообщениями, используя, согласно спецификациям FIPA, соответствующий язык содержания (язык SLO) и соответствующую онтологию (онтологию FIPA-agent-management). Чтобы упростить эти взаимодействия, JADE предоставляет класс jade.domain.DFService, с помощью которого можно публиковать и выполнять сервисы через вызовы методов.

Публикация сервисов

Агент, желающий опубликовать (сделать доступными) один или более сервисов, должен предоставить DF, включающий AID этого агента, список возможных языков и онтологий, которые должны быть известны другим агентам для взаимодействия с ним, и список сервисов для публикации. Для каждого публикуемого сервиса предоставляется описание, включающее тип сервиса, его имя, языки и онтологии, необходимые для его использования, и еще некоторые свойства, специфичные для данного сервиса. Классы ***DFAgentDescription***, ***ServiceDescription*** и ***Property***, входящие в состав пакета ***jade.domain.FIPAAgentManagement***, соответственно представляют собой три упомянутые абстракции.

Чтобы опубликовать сервис, агент должен создать описание, представленное экземпляром класса ***DFAgentDescription***, и вызвать статический метод ***register()*** класса ***DFService***. Обычно регистрация сервиса выполняется в методе ***setup()***, как показано ниже в примере с агентом-продавцом книг.

```
    protected void setup() {
...
// Register the book-selling service in the yellow pages
DFAgentDescription dfd = new DFAgentDescription()
dfd.setName(getAID());
ServiceDescription sd = new ServiceDescription();
sd.setType("book-selling");
sd.setName("JADE-book-trading");
dfd.addServices(sd);
try {
DFService.register(this, dfd);
```

```
    }
    catch (FIPAException fe) {
    fe.printStackTrace();
  }
...
}
```

Заметим, что в этом простом примере не были описаны ни языки, ни онтологии, ни свойства, специфичные для данного сервиса. Когда агент завершает работу, следует отменить регистрацию опубликованных сервисов.

```
protected void takeDown() {
// Deregister from the yellow pages
try {
DFService.deregister(this);
}
catch (FIPAException fe) {
fe.printStackTrace();
}
// Close the GUI
myGui.dispose();
// Printout a dismissal message
System.out.println("Seller-agent "+getAID().getName()+" terminating.");
}
```

Поиск сервисов

Агент, которому требуется найти некоторые сервисы, должен предоставить агенту DF описание необходимого шаблона. Результатом поиска является список всех описаний, которые удовлетворяют указанному шаблону.

Описание соответствует шаблону, если все поля, указанные в шаблоне, присутствуют с теми же значениями и в описании. Использование статического метод *search()* класса *DFService* можно проиллюстрировать примером, в котором агент-покупатель книг динамически находит всех агентов, предоставляемых сервисом *«book-selling»* (продажа книг).

```
 public class BookBuyerAgent extends Agent {

private String targetBookTitle; // The title of the book to buy
private AID[] sellerAgents; // The list of known seller agents

// Put agent initializations here
protected void setup() {
// Printout a welcome message
System.out.println("Hello! Buyer-agent "+getAID().getName()+" is ready.");
// Get the title of the book to buy as a start-up argument
Object[] args = getArguments();
if (args != null && args.length > 0) {
targetBookTitle = (String) args[ 0];
System.out.println("Trying to buy "+targetBookTitle);
// Add a TickerBehaviour that schedules a request to seller agents every minute

addBehaviour(new TickerBehaviour(this, 60000) {
 protected void onTick() {
 // Update the list of seller agents
DFAgentDescription template = new DFAgentDescription();
ServiceDescription sd = new ServiceDescription();
sd.setType("book-selling");
template.addServices(sd);
       try {
       DFAgentDescription[] result = DFService.search(myAgent, template);
       sellerAgents = new AID[result.length];
       for (int i = 0; i < result.length; ++i) {
       sellerAgents[i] = result[i].getName();
       }
```

```
        catch (FIPAException fe) {
        fe.printStackTrace();
        }
     // Perform the request
    myAgent.addBehaviour(new RequestPerformer());
    }
    } );
    ...
```

Заметим, что обновление списка известных агентов-продавцов производится каждый раз перед очередной попыткой купить указанную книгу, т.к. агенты-продавцы в системе могут динамически появляться и исчезать.

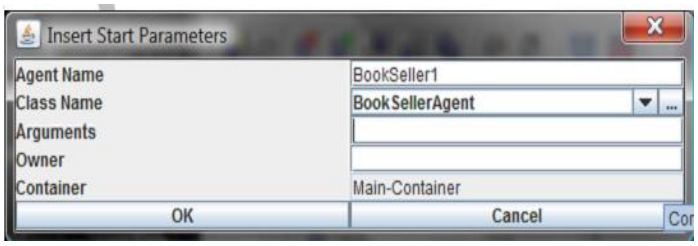
Класс *DFService* также предоставляет поддержку подписки на уведомления от DF о появлении агента, предоставляющего указанный сервис (методы *searchUntilFound()* и *createSubscriptionMessage()*).

Ход работы:

1. Создать новый проект (например, **bookTrading**), сформировав код в виде 3-х классов: **BookSellerAgent, BookByerAgent, BookSellerGui** в **JADE\src\examples\bookTrading.**
2. Скомпилировать и запустить проект, использовав строку: **-gui –agents seller:<имя пакета>.BookSellerAgent; byer:<имя пакета>.BookBuyerAgent1(Java_programming)**

Примечание: наименования книг в форме создания не должны содержать пробелов, т.к. система обрезает наименования до первого пробела и, соответственно, искомая книга найдена не будет (хотя агент продавца и позволяет вводить наименования книг с пробелами). Поэтому пробелы должны заменяться в наименовании, например, нижним подчеркиванием.

3. Создайте несколько агентов-продавцов:

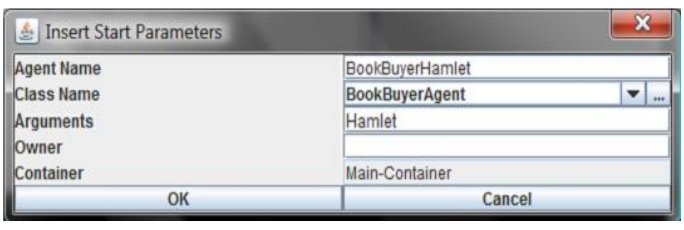


На экране появится форма для ввода наименования книги и ее цены, после каждого нажатия на кнопку <OK> будет создано новое наименование книги с заданными в форме параметрами для агента продавца, имя которого указано в заголовке формы:



Примечание: при закрытии формы агент прекращает работу! Поэтому формы ввода параметров для всех агентов продавцов должны все время быть открытыми!

4. Создайте агента-покупателя книг. Наименование книги, которую хочет купить агент, указывается в поле Arguments:



Примечание: наименования книг в форме создания не должны содержать пробелов, т.к. система обрезает наименования до первого пробела и, соответственно, искомая книга найдена не будет (хотя агент продавца и позволяет вводить

наименования книг с пробелами). Поэтому пробелы должны заменяться в наименовании, например, нижним подчеркиванием.

После создания агент регистрируется в системе и сразу же начинает выполнять матчинг для поиска соответствия наименования требуемой книги и наименований предлагаемых книг. Агент покупателя посылает запросы агентам продавцов периодически через определенный интервал времени.

После регистрации всех агентов покупателей они сразу вступают в процесс матчинга. Каждый агент покупателя находит наиболее подходящего продавца (предлагающего наименьшую цену за искомую книгу) и создает с ним связь.

ЗАДАНИЕ К ЛРN№2

1. Пронаблюдайте результат взаимодействия агентов, в том числе в сниффере.
2. Сделайте выводы.
3. Внимательно изучите код особенно в части поведения агентов.
4. Создайте несколько агентов-продавцов и покупателей.
5. Проследите процесс взаимодействия и результат, в том числе в сниффере.
6. Сделайте выводы.
7. Внимательно изучите код в части обмена сообщениями.
8. Приступите к выполнению индивидуального задания (Приложение А).